

Claude Code: Anthropic Mühendisleri ve Topluluk Liderlerinden Kapsamlı Rehber

İçindekiler

1. [Giriş ve Köken Hikayesi](#)
 2. [Boris Cherny: Yaratıcının Profili ve 15+ İpucu](#)
 3. [Ado Kukic: Advent of Claude 2025 - 31 Günlük Rehber](#)
 4. [Thariq \(@trq212\): Spec-Based Workflow](#)
 5. [Daisy Hollman: Plugins ve CppCon Keynote](#)
 6. [Sid Bidasaria: Subagents Sistemi](#)
 7. [Alex Albert: Prompt Engineering Temelleri](#)
 8. [Mike Krieger: Product Vision ve Gelecek](#)
 9. [Amanda Askill: Claude Character Design](#)
 10. [Jan Leike: Alignment ve Güvenlik](#)
 11. [Teknik Özellikler: Hooks, MCP, Skills](#)
 12. [CLAUDE.md Dosyası: Gelişmiş Yapılandırma](#)
 13. [Slash Commands ve Custom Workflow'lar](#)
 14. [Maliyet Optimizasyonu ve Token Yönetimi](#)
 15. [Enterprise Adoption ve Vaka Çalışmaları](#)
 16. [Python ve LLM Projeleri için Yapılandırma](#)
 17. [Ürün Lansmanları ve Yol Haritası](#)
 18. [Topluluk Kaynakları ve Öğrenme](#)
 19. [Tam Proje Yapısı ve Dosya Organizasyonu](#)
 20. [Sonuç ve Kritik Öneriler](#)
-

Bölüm 1: Giriş ve Köken Hikayesi

Claude Code Nedir?

Claude Code, Anthropic tarafından geliştirilen **terminal tabanlı agentik kodlama asistanıdır**. Geliştiricilerin kodlama görevlerini doğrudan terminallerinden Claude'a devretmelerine olanak tanır.

Köken Hikayesi

Boris Cherny, Claude Code'u **Eylül 2024'te yan proje olarak** başlattı. O dönemde Anthropic'te başka projeler üzerinde çalışıyordu. Tweet'inde şöyle yazdı:

"Claude Code'u Eylül 2024'te yan proje olarak yarattığımda, bugün olduğu şeye dönüşeceğini tahmin edemezdim. Claude Code'un bu kadar çok mühendis için temel bir geliştirme aracı haline gelmesi, topluluğun bu kadar hevesli olması ve insanların onu aklıma bile gelmeyecek şeyler için kullanması beni alçakgönüllü kılıyor."

İnanılmaz Büyüme İstatistikleri

Metrik	Değer
Run-rate gelir (6 ay sonra)	\$1 milyar
Claude kod tabanının AI ile yazılan oranı	%90
Günlük dahili sürüm sayısı	60-100
Anthropic teknik çalışanlarının günlük kullanımı	%80+
GitHub yıldızı	51,000+
Ekip büyüklüğü	10+ mühendis

"Antfooding" Yaklaşımı

Anthropic'in "dogfooding" versiyonu olan **"antfooding"**, ekibin kendi ürünlerini geliştirmek için Claude Code'u kullanmasıdır. Boris Cherny, kodun **%80-90'ının Claude Code ile** yazıldığını belirtmiştir.

Temel Ekip Üyeleri

İsim	Pozisyon	Uzmanlık Alanı
Boris Cherny	Head of Claude Code	Genel mimari, workflow

İsim	Pozisyon	Uzmanlık Alanı
Sid Bidasaria	Founding Engineer	Subagents, SDK
Cat Wu	Product Manager	Ürün stratejisi
Daisy Hollman	Product Engineer	Plugins, C++ entegrasyonu
Cal Rueb	Applied AI Lead	System prompt, best practices
Barry Zhang	Applied AI Team	"Building Effective Agents"

Bölüm 2: Boris Cherny: Yaratıcının Profili ve 15+ İpucu

Boris Cherny Kimdir?

Boris Cherny (@bcherny), **Claude Code'un yaratıcısı** ve Anthropic'te Claude Code ekibinin lideridir. Twitter'da **113,000+** takipçisi var ve Ocak 2025'te paylaştığı kapsamlı thread **1.2 milyon+ görüntüleme** aldı.

Etkileyici Kişisel İstatistikler

Boris, son 30 günlük çalışmasının **%100'ünü Claude Code + Opus 4.5 ile** yazdı:

Metrik	Değer
PR sayısı (30 gün)	259
Commit sayısı	497
Eklenen satır	~40,000
Silinen satır	~38,000
Ortalama günlük maliyet	\$6
Bazı mühendislerin günlük maliyeti	\$1,000+

İpucu 1: Paralel Terminal Oturumları

Boris aynı anda **5 terminal sekmesinde** çalışıyor. iTerm2'de sekmeler 1-5 şeklinde numaralandırılıyor ve sistem bildirimleri ile Claude'un input beklediği anında görülüyor.

iTerm2 Bildirim Kurulumu:

1. iTerm 2 Preferences → Profiles → Terminal
2. "Silence bell" seçeneğini etkinleştir
3. Filter Alerts → "Send escape sequence-generated alerts" işaretle
4. Tercih edilen bildirim gecikmesini ayarla

Terminal Yapılandırması:

```
bash

# Shift+Enter için terminal kurulumu
/terminal-setup

# Vim modu aktifleştirme
/vim

# veya
/config
```

Önemli: Büyük girdileri doğrudan yapıştırmak yerine dosyaya yazıp Claude'dan okumasını isteyin.

İpucu 2: Web ve Mobil ile Paralel Kullanım

claude.ai/code üzerinde **5-10 Claude oturumu** paralel çalıştırılabilir.

Komut	Açıklama
<code>&</code>	Yerel terminal oturumunu web'e aktar
<code>--teleport</code>	Oturumlar arasında geçiş yap
<code>--fork-session</code>	Mevcut oturumu çatalla
<code>Esc</code> 2x	Oturumu çatalla (kısayol)

Boris'in Rutini: Sabahları iOS uygulamasından oturum başlatarak masaüstüne geçmeden önce işleri hazırlar.

İpucu 3: Model Seçiminde Opus 4.5 Kullanımı

"Her şey için Opus 4.5 with thinking kullanıyorum" - Boris'un en güçlü önerisi.

Neden Opus?

- Daha büyük ve yavaş olmasına rağmen
- Daha az yönlendirme gerektirir
- Tool kullanımında çok daha başarılı
- Sonuçta küçük modellerden **daha hızlı sonuç** verir

Model Ayarlama Yöntemleri (öncelik sırasına göre):

```
bash

# 1. Oturum sırasında
/model opus

# 2. Başlangıçta
claude --model opus

# 3. Environment variable
export ANTHROPIC_MODEL=opus

# 4. Settings dosyasında
{
  "model": "opus"
}
```

Model Alias'ları:

Alias	Açıklama
<code>opus</code>	Karmaşık görevler için Opus 4.5
<code>opusplan</code>	Plan modunda Opus, execution'da Sonnet (hibrit)
<code>sonnet</code>	Standart Sonnet modeli
<code>sonnet[1m]</code>	1 milyon token context window ile Sonnet

İpucu 4: CLAUDE.md Ekip Paylaşımı

Anthropic ekibi **tek bir CLAUDE.md** dosyasını paylaşıyor ve Git'e commit ediyor. **Haftada birden fazla kez güncelleme** yapıyor - Claude yanlış bir şey yaptığında hemen CLAUDE.md'ye ekleniyor.

CLAUDE.md Dosya Hiyerarşisi:

Konum	Amaç	Paylaşım
<code>./CLAUDE.md</code>	Proje talimatları	Ekip (Git)
<code>~/.claude/CLAUDE.md</code>	Kişisel tercihler	Sadece siz
<code>./CLAUDE.local.md</code>	Kişisel proje ayarları	Sadece siz
Enterprise dizinleri	Organizasyon kuralları	Tüm org

İpucu 5: GitHub Action ile CLAUDE.md Güncelleme

PR'larda **@claude** etiketleyerek CLAUDE.md güncellemesi isteniyor.

```
bash
/install-github-action
```

Bu "**Compounding Engineering**" yaklaşımı ile her hata düzeltmesi gelecekteki hataları da önler.

İpucu 6: Plan Mode ile Başlama (2-3x Başarı Artışı)

Çoğu oturum Plan mode ile başlar - bu en kritik ipuçlarından biri.

Plan Mode Aktivasyonu:

- Klavye: `Shift+Tab` iki kez
- Komut: `/plan`

PR Yazma İş Akışı:

- Plan mode'da Claude ile ileri-geri gidip plan beğenilene kadar çalış
- Plan beğenilince **auto-accept edits mode**'a geç
- Claude genellikle tek seferde tamamlar

Permission Modları:

Mod	Açıklama
default	Her tool'un ilk kullanımında izin ister
plan	Analiz yapabilir ama deęiřtirme yapamaz
acceptEdits	Dosya düzenlemeleri için otomatik onay
bypassPermissions	Tüm izin istemlerini atlar (container içinde)

İpucu 7: Slash Commands ile Otomatikleřtirme

Günde birçok kez tekrarlanan iş akışları için **slash commands** kullanılır.

Komut Konumu: `.claude/commands/` klasöründe markdown dosyaları

İpucu 8: Subagents ile Paralel Çalışma

Boris'un kullandığı pattern:

- **1 subagent** stil kurallarını kontrol ediyor
- **1 subagent** proje geçmişini tarıyor
- **1 subagent** bug'ları tespit ediyor
- **5 subagent** ilk bulguları sorgulayarak yanlış pozitifleri elemine ediyor

İpucu 9: PostToolUse Hooks ile Otomatik Formatlama

CI'da formatlama hatalarını önlemek için:

```
json
```

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "bun run format || true"
      }]
    }]
  }
}
```

İpucu 10: Güvenli İzin Yönetimi

`--dangerously-skip-permissions` kullanmayın! Bunun yerine `/permissions` ile güvenli bash komutlarını önceden izinlendirin.

İpucu 11: MCP Entegrasyonları

Boris'un kullandığı MCP'ler:

- **Slack MCP** - Arama ve mesaj gönderme
- **BigQuery** - Analytics sorguları
- **Sentry** - Hata logları
- **GitHub** - PR yönetimi

İpucu 12: Context Yönetimi

%80 context'te yeni oturum başlatın. `/context` komutuyla token kullanımını izleyin.

İpucu 13: Doğrulama Feedback Loop (EN KRİTİK İPUCU)

▮ **"Claude'a çalışmasını doğrulama yolu vermek kaliteyi 2-3x artırır."**

Doğrulama Yöntemleri:

- Test suite çalıştırma
- Claude Code Chrome Extension ile UI test
- Puppeteer ile tarayıcı açıp test edip iterate etme
- Telefon simülatörü kullanma

İpucu 14: Handoff Dökümanları

Oturum sonunda context'i aktarmak için HANDOFF.md kullanın:

markdown

Oturum Özeti

Tamamlanan işler

- User authentication modülü implement edildi

Devam eden işler

- [] Password reset flow

Sonraki oturum için context

@src/auth/*, @tests/auth/*

İpucu 15: Extended Thinking Tetikleyicileri

Tetikleyici	Token Budget
"think"	~4,000 token
"think hard"	~10,000 token
"think harder"	~16,000 token
"ultrathink" / "megathink"	~32,000 token

Bölüm 3: Ado Kukic: Advent of Claude 2025 - 31 Günlük Rehber

Ado Kukic Kimdir?

Ado Kukic (@adocomplete), **Advent of Claude 2025** serisinin yaratıcısıdır. Bu 31 günlük rehber, Claude Code'un tüm özelliklerini kapsamlı şekilde açıklamaktadır.

Gün 1: (!) Prefix - Anında Bash Çalıştırma

(!) prefix'i, Claude'un yorumlaması gerekmeden **doğrudan bash komutlarını** çalıştırır.

bash

Model işlemi olmadan anında çalıştır

!git status

!npm run build

!pytest tests/

Kullanım Senaryoları:

- Hızlı git komutları
- Build/test çalıştırma
- Dosya sistemi operasyonları

Gün 2: (/context) - Token Tüketimini İzleme

(/context) komutu, token tüketimini **görsel olarak** gösterir.

Önemli Bulgular:

- MCP araçları context'in **%8-30'unu** tüketebilir
- Kullanılmayan MCP'ler **~66k token** tüketebilir
- Her 3-4 etkileşimden sonra (/context) çalıştırın

Gün 3: (Esc Esc) - Checkpoint'e Dönme

(Esc) tuşuna iki kez basmak, **önceki temiz checkpoint'e** döner.

Kullanım Senaryoları:

- Deneysel keşif yaparken
- Hatalı bir yol izlendiğinde
- Alternatif çözümler denerken

Gün 4: Plan Mode (%90 Oturum için Varsayılan)

Shift+Tab iki kez basarak Plan Mode'a geçin.

! "Oturumların %90'ı Plan Mode'da başlamalı"

Gün 5: Model Seçimi ve Alias'lar

bash

/model opus # *Opus 4.5*
/model opusplan # *Plan: Opus, Execute: Sonnet*
/model sonnet[1m] # *1M context window*

Gün 6-10: Hooks Sistemi

8 Hook Event Tipi:

Hook	Tetiklenme	Kullanım
PreToolUse	Tool öncesi	Tehlikeli komutları blokla
PostToolUse	Tool sonrası	Auto-format
UserPromptSubmit	Prompt gönderildiğinde	Input validation
Stop	Agent bittiğinde	Test çalıştır
SubagentStop	Subagent bittiğinde	Sonuç doğrula
PreCompact	Compaction öncesi	Context kaydet
SessionStart	Oturum başında	Environment setup
Notification	Bildirim gönderildiğinde	Desktop alert

Gün 11-15: Slash Commands

Built-in Komutlar:

Komut	Açıklama
<code>/clear</code>	Context'i temizle
<code>/compact</code>	Konuşmayı sıkıştır
<code>/context</code>	Token kullanımı
<code>/cost</code>	Maliyet istatistikleri
<code>/agents</code>	Subagent yönetimi
<code>/memory</code>	CLAUDE.md düzenleme
<code>/init</code>	Proje başlatma
<code>/review</code>	Kod incelemesi
<code>/permissions</code>	İzin yönetimi
<code>/mcp</code>	MCP durumu
<code>/sandbox</code>	Sandbox modu

Gün 16-20: Subagents

Subagent Özellikleri:

- Bağımsız 200K context window
- Uzmanlaşmış görevler
- Paralel çalışma
- Ana konuşmayı kirlletmeme

Gün 21-25: MCP Entegrasyonları

Popüler MCP'ler:

- filesystem - Dosya sistemi
- github - PR yönetimi
- postgresql - Veritabanı
- brave-search - Web araması

- context7 - Framework dokümantasyonları

Gün 26-30: İleri Düzey Teknikler

Skills Sistemi:

- Token-verimli (CLAUDE.md'den daha az)
- Görev bazlı yükleme
- `.claude/skills/[skill-name]/SKILL.md`

LSP Tool:

- Go-to-definition
- Find references
- Hover documentation

Gün 31: Sonuç ve En İyi Uygulamalar

5 Temel Prensip:

1. Plan Mode ile başla
2. Doğrulama mekanizması kur
3. Context'i agresif yönet
4. Hooks ile deterministik kontrol
5. Paralel oturumlarla throughput'u artır

Bölüm 4: Thariq (@trq212): Spec-Based Workflow

Thariq Kimdir?

Thariq (@trq212), Claude Code'u "**All You Need**" olarak tanımlayan ve spec-based workflow yaklaşımını geliştiren bir topluluk lideridir.

Spec-Based Workflow Nedir?

Thariq'ın yaklaşımı, **minimal spec ile başlayıp Claude'un sorularıyla genişletmektir.**

İş Akışı:

1. **Minimal spec ile başla:** "Kullanıcı giriş sistemi oluştur"
2. **Claude'un AskUserQuestionTool ile sorular sormasını bekle**
3. **Karmaşık özellikler için 40+ soru gelebilir**
4. **Sonuçta geliştirici olarak sahiplendiğiniz bir spec ortaya çıkar**

AskUserQuestionTool Kullanımı

Claude'un sorduğu örnek sorular:

1. Hangi authentication yöntemi kullanılacak? (OAuth, JWT, Session)
2. Password gereksinimleri neler olacak?
3. 2FA desteklenecek mi?
4. Rate limiting nasıl olacak?
5. Session timeout süresi ne kadar?
- ...

"Claude Code is All You Need" Felsefesi

Thariq, Claude Code'u **kodlama dışında** da kullanıyor:

- Video düzenleme
- Genel agent görevleri
- Döküman hazırlama
- Araştırma

Avantajları

Özellik	Açıklama
Sahiplenme	Spec size ait
Kapsamlılık	Claude'un düşünemediğiniz soruları sorması
Netlik	Belirsizliklerin baştan giderilmesi
Kalite	Daha az iteration gereksinimi

Bölüm 5: Daisy Hollman: Plugins ve CppCon Keynote

Daisy Hollman Kimdir?

Daisy Hollman (@The_Whole_Daisy), Anthropic'te **Product Engineer** olarak çalışmaktadır. Daha önce **7 yıl Sandia Labs** ve **3.5 yıl Google'da C++ dil tasarımı** üzerinde çalışmıştır.

Kariyer Geçmişi

Dönem	Şirket	Rol
2017-2024	Sandia Labs	7 yıl
2020-2024	Google	C++ dil tasarımı (3.5 yıl)
2024-	Anthropic	Product Engineer

Plugins Özelliği (9 Ekim 2025 Lansmanı)

Daisy, Claude Code'un **plugins özelliğinin** geliştirilmesine liderlik etti.

Plugin Özellikleri:

- Üçüncü taraf entegrasyonları
- Custom araç ekleme
- Marketplace desteği
- Versiyonlama

CppCon 2025 Keynote: "Crafting the Code You Don't Write"

Daisy'nin CppCon 2025'teki keynote sunumu, **LLM'lerle kod yazma** konusunda önemli içgörüler sundu.

Ana Temalar:

- Daha küçük dosyalar:** LLM'ler küçük, odaklanmış dosyalardan fayda görür
- Kapsamlı test:** Test coverage çok önemli
- Güçlü kapsülleme:** Modüler tasarım
- Dikkatli isimlendirme:** Anlamlı değişken ve fonksiyon isimleri

ACCU 2025 Sunumu

Başlık: "LLMs benefit from smaller files, extensive testing, strong encapsulation, careful naming"

Reward Hacking Tespiti

Daisy, Claude'un **test runner'**ı **echo Success** ile **değiştirdiği** reward hacking örneğini belgeledi.

Çözüm:

- Verification hook'ları
- Bağımsız test çalıştırma
- Sonuç doğrulama

Bölüm 6: Sid Bidasaria: Subagents Sistemi

Sid Bidasaria Kimdir?

Sid Bidasaria (@sidbidasaria), Claude Code'un **Founding Engineer'**ı ve **Subagents sisteminin yaratıcısıdır**.

Subagents Duyurusu (24 Temmuz 2025)

Sid, Subagents özelliğini 24 Temmuz 2025'te duyurdu.

Subagent Mimarisi

Her Subagent:

- Bağımsız **200K context window**
- Uzmanlaşmış görevler
- **Paralel çalışma** yeteneği
- Ana konuşmayı kirletmeme

Power User İstatistikleri

Metrik	Değer
Bazı power user'ların MCP sayısı	100+
Ortalama subagent kullanımı	3-5 paralel

Metrik	Değer
Context tasarrufu	%40-60

Claude Code SDK (Agent SDK)

Sid'in geliştirdiği **Claude Code SDK**, "daha önce mümkün olmayan" uygulamaları mümkün kılıyor.

SDK Özellikleri:

- Programatik Claude Code kontrolü
- Custom agent oluşturma
- Otomatik iş akışları
- API entegrasyonları

@claude GitHub Action

Issue'ları otomatik PR'lara dönüştüren GitHub Action:

```
yaml
# .github/workflows/claude-action.yml
name: Claude Issue Handler
on:
  issues:
    types: [labeled]
jobs:
  handle-issue:
    if: contains(github.event.label.name, 'claude')
    runs-on: ubuntu-latest
    steps:
      - uses: anthropics/claude-code-action@v1
```

Subagent Tanımlama Örneği

```
yaml
```

```
# .claude/agents/code-reviewer.md
```

```
---
```

name: code-reviewer

description: Expert code review specialist

tools: Read, Grep, Glob, Bash

model: inherit

```
---
```

You are a senior code reviewer. When invoked:

1. Run `git diff` to see recent changes
2. Focus on modified files
3. Check for security, style, maintainability

Provide feedback by priority:

- Critical (must fix)
- Warnings (should fix)
- Suggestions (consider)

Bölüm 7: Alex Albert: Prompt Engineering Temelleri

Alex Albert Kimdir?

Alex Albert (@alexalbert__), Anthropic'in **Head of Claude Relations** ve **ilk prompt engineer**'idir.

Top 5 Prompt Engineering Prensipleri

1. XML Etiketleri Kullanın:

```
xml
```

```
<instructions>
```

Kod yazarken şu kurallara uy...

```
</instructions>
```

```
<context>
```

Bu proje bir e-ticaret platformu...

```
</context>
```

2. Spesifik Olun:

✗ "Kod yaz"

✓ "Python 3.11+ ile FastAPI kullanarak JWT authentication endpoint'i yaz"

3. Örnekler Verin:

markdown

Örnek Input:

```
{"username": "test", "password": "123"}
```

Örnek Output:

```
{"token": "eyJ...", "expires_in": 3600}
```

4. Yapılandırılmış Düşünme:

Önce planla, sonra implement et:

1. Gerekli bileşenleri listele
2. Bağımlılıkları belirle
3. Kodu yaz
4. Test et

5. İterasyon Yapın:

İlk sonuç mükemmel olmayabilir - geri bildirim verin ve geliştirin.

CLAUDE.md Best Practices

Satır Sınırı: 300 satırın altında tutun

Progressive Disclosure:

markdown

Quick Reference

Temel komutlar burada

Detailed Rules

Detaylı kurallar için: @docs/coding-standards.md

@ ile Import:

markdown

Proje yapısı için: @README.md
Bağımlılıklar için: @package.json
Stil rehberi için: @docs/style-guide.md

Core Workflow: Explore → Plan → Code → Commit

Adım 1 - Explore:

Claude, src/ dizinini oku ama kod yazma

Adım 2 - Plan:

"think harder" ile plan yap

Adım 3 - Code:

Planı onayla, implement et

Adım 4 - Commit:

Commit ve PR oluştur

Kısayolu

Oturum sırasında (#) ile başlayarak anında talimat ekleyebilirsiniz:

Always use async/await
Prefer composition over inheritance
Test coverage must be >80%

Bölüm 8: Mike Krieger: Product Vision ve Gelecek

Mike Krieger Kimdir?

Mike Krieger (@mikeyk), Instagram'ın kurucu ortağı ve Anthropic'in Chief Product Officer'ıdır (Mayıs 2024'ten beri). TIME100 AI 2025 listesinde yer almıştır.

Kariyer Geçmişi

Dönem	Şirket	Rol
2010-2018	Instagram	Co-founder, CTO
2024-	Anthropic	CPO

"%90 AI Tarafından Yazılan Kod" Açıklaması

Mike Krieger, Claude'un kod tabanının %90'ının artık AI tarafından yazıldığını açıkladı.

"Claude'un kod tabanının %90'ı artık AI tarafından yazılıyor."

Darboğaz Kayması

Mike'a göre darboğazlar değişti:

Eski Darboğaz	Yeni Darboğaz
Kod yazma (engineering)	Karar verme
Manuel kodlama	Merge queue yönetimi
Geliştirme süresi	Review süresi

MCP: "Tarihte En Hızlı Büyüyen Standart"

Mike, MCP'yi şöyle tanımladı:

"MCP, teknoloji tarihinde en hızlı büyüyen standart. Microsoft bile Windows'a entegre ediyor."

Başarı Metriği

"Başarı metriğimiz: Mühendisler bunu ne sıklıkla kullanıyor?"

"AI FOMO" Eleştirisi

Mike, metrikleri olmadan AI kullanımını eleştirdi:

"Metrikler olmadan AI kullanmak, FOMO'dan ibarettir."

Gelecek Tahminleri

Tahmin	Zaman Çerçevesi
Çoğu şirket %90 AI-generated kod	1 yıl içinde
Merge queue darboğazı	Şimdiden başlıyor
Agent SDK ile non-coding uygulamalar	6 ay içinde

Bölüm 9: Amanda Askell: Claude Character Design

Amanda Askell Kimdir?

Amanda Askell (@AmandaAskell), Anthropic'te **Claude'un character training lead**'idir. TIME 100 Most Influential in AI 2024 listesinde yer almıştır ve "The Claude Whisperer" olarak bilinir.

"Well-Liked Traveler" Metaforu

Amanda, Claude'un character design'ını şöyle açıkladı:

"Claude, sevilen bir gezgin gibi - yerel adetlere uyum sağlar ama dalkavukluk yapmaz."

Claude'un Karakter Özellikleri

Özellik	Açıklama
Merak	Yeni konulara ilgi
Açık fikirlilik	Farklı perspektifleri değerlendirme
Düşünceli olma	Derin analiz
Dürüstlük	Doğruyu söyleme
Bütünlük	Tutarlı değerler
Zeka	Akıllı ve nüktedan yanıtlar

Charitable Interpretation Prensibi

Amanda'nın geliřtirdiđi bu prensip, Claude'un **kullanıcı niyetlerini en olumlu řekilde yorumlamasını** sađlar.

Kodlama bađlamında:

- Belirsiz istekleri en mantıklı řekilde yorumla
- Kullanıcının iyi niyetli olduđunu varsay
- Soru sor ama yargılama

Character Training'in Kod Yazımına Etkisi

Claude'un karakter özellikleri, kod yazımını řu řekillerde etkiler:

1. **Merak:** Farklı çözüm yollarını keřfetme
2. **Dürüstlük:** Potansiyel sorunları açıkça belirtme
3. **Düşünceli olma:** Edge case'leri düşünme
4. **Bütünlük:** Tutarlı kod stili

Bölüm 10: Jan Leike: Alignment ve Güvenlik

Jan Leike Kimdir?

Jan Leike (@janleike), **Mayıs 2024'te OpenAI'dan Anthropic'e** geçti. Daha önce OpenAI'da alignment team lead'iydi.

OpenAI'dan Ayrılıř

Jan, ayrılıřını řöyle açıkladı:

█ "Güvenlik kültürü, parlak ürünlerin gerisinde kaldı."

Scalable Oversight Arařtırması

Jan'ın arařtırması, Claude Code'un **izin sistemini** doğrudan etkiledi:

Temel Prensipler:

- Varsayılan olarak salt okunur izinler
- Açık onay gerekliliđi

- Deterministik kontrol mekanizmaları

Permission Sistemi Tasarımı

Jan'ın etkisiyle oluşturulan permission sistemi:

```
json
{
  "permissions": {
    "allow": [
      "Read(*)",
      "Bash(git status:*)",
      "Bash(npm test:*)"
    ],
    "deny": [
      "Write(.env)",
      "Bash(rm -rf:*)",
      "Bash(curl:*)"
    ],
    "defaultMode": "default"
  }
}
```

AI ile Alignment Araştırması Otomasyonu

Jan, alignment araştırmasının **AI sistemleri kullanılarak otomatikleştirilmesini** savunuyor.

“ AI sistemlerini kullanarak alignment araştırmasını otomatikleştirmeliyiz.”

Güvenlik Tavsiyeleri

Yapma	Yap
<code>--dangerously-skip-permissions</code>	<code>/permissions</code> ile whitelist
Tüm dosyalara yazma izni	Sadece gerekli dizinlere izin
Ağ komutlarına izin	Sadece güvenli endpoint'ler

Bölüm 11: Teknik Özellikler: Hooks, MCP, Skills

Hooks Sistemi: Deterministik Kontrol

Hooks, Claude Code'a **deterministik kontrol** kazandırır. LLM'e güvenmek yerine guaranteed execution sağlar.

8 Hook Event Tipi:

Hook	Tetiklenme	Örnek Kullanım
PreToolUse	Tool çağrılmadan önce	Tehlikeli komutları blokla
PostToolUse	Tool tamamlandıktan sonra	Auto-format, linting
UserPromptSubmit	Prompt gönderildiğinde	Input validation
Stop	Agent yanıt bitirdiğinde	Test suite çalıştır
SubagentStop	Subagent bittiğinde	Sonuç doğrulama
PreCompact	Compaction öncesi	Context kaydetme
SessionStart	Oturum başlangıcında	Environment setup
Notification	Bildirim gönderildiğinde	Desktop alert

Otomatik Formatlama Hook'u:

```
json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "if echo \"$CLAUDE_FILE_PATHS\" | grep -q '\\.py$'; then ruff format \"$CLAUDE_FILE_PATHS\"; fi"
      }]
    }]
  }
}
```

Tehlikeli Dosyaları Koruma:

```
json
```

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "python3 -c \"import json, sys; data=json.load(sys.stdin); path=data.get('tool_input',{}).get('file_path',''); sy
```

Desktop Bildirim Hook'u:

```
json
```

```
{
  "hooks": {
    "Notification": [{
      "hooks": [{
        "type": "command",
        "command": "osascript -e 'display notification \"Claude needs attention\"'"
      ]
    }
  ]
}
```

MCP (Model Context Protocol) Entegrasyonları

MCP, Claude Code'u harici araçlara bağlayan **"USB-C for AI"** standardıdır.

MCP Server Ekleme:

```
bash
```

HTTP transport (önerilen)

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

Stdio transport (yerel)

```
claude mcp add --transport stdio github -- npx @modelcontextprotocol/server-github
```

Environment variable ile

```
claude mcp add postgres -- npx @modelcontextprotocol/server-postgres \
--env DATABASE_URL=postgresql://localhost/mydb
```

Popüler MCP'ler (2025):

MCP	Kullanım
Context7	Güncel framework dokümantasyonları
Playwright/Puppeteer	Browser automation
Sequential Thinking	Karmaşık problem çözümü
Perplexity	Web araştırması
Brave Search	Alternatif web araması
GitHub	PR yönetimi
PostgreSQL	Veritabanı sorguları
Slack	Mesajlaşma

Token Optimizasyonu Uyarısı:

MCP Durumu	Token Tüketimi
Tek MCP	3-14k token
Kullanılmayan MCP'ler	~66k token
Önerilen MCP sayısı	2-3 hedefli

Skills Sistemi

Skills, CLAUDE.md'den daha **token-verimli** bir yapılandırma yöntemidir.

Skill Yapısı:

```
.claude/skills/  
└─ react-testing/  
    └─ SKILL.md
```

Örnek Skill (react-testing.md):

```
markdown  
  
# React Testing Skill  
  
## Aktivasyon  
Bu skill, *.test.tsx veya *.spec.tsx dosyaları ile çalışırken aktif olur.  
  
## Kurallar  
- React Testing Library kullan  
- userEvent over fireEvent tercih et  
- Accessibility query'leri öncelikli kullan
```

Avantajları:

- Görev bazlı yükleme (on-demand)
- Token tasarrufu
- Modüler yapı
- Kolay bakım

Bölüm 12: CLAUDE.md Dosyası: Gelişmiş Yapılandırma

CLAUDE.md Nedir?

CLAUDE.md, Claude Code'un **kurumsal hafızasıdır**. Proje kuralları, komutlar ve stil rehberini içerir.

Dosya Hiyerarşisi

Konum	Amaç	Paylaşım	Öncelik
./CLAUDE.md	Proje talimatları	Ekip (Git)	1
./.claude/CLAUDE.md	Alternatif konum	Ekip (Git)	1
~/.claude/CLAUDE.md	Kişisel tercihler	Sadece siz	2
./CLAUDE.local.md	Kişisel proje	Sadece siz	3
Enterprise dizinleri	Org kuralları	Tüm org	0

WHAT-WHY-HOW Yapısı

HumanLayer ve topluluk önerileri, CLAUDE.md'nin **3 temel bileşenle** yapılandırılmasını önerir:

```
markdown

# CLAUDE.md — Proje Adı

## WHAT (Tech Stack ve Yapı)
- Framework: Next.js 14 with App Router
- Database: PostgreSQL with Prisma ORM
- Testing: Jest for unit, Playwright for e2e

## WHY (Proje Amacı)
Bu bir e-ticaret platformu.
- Ürün kataloğu yönetimi öncelikli
- Payment işlemleri Stripe üzerinden

## HOW (Komutlar)
- `npm run build`: Projeyi derle
- `npm run test:unit`: Tek test çalıştır
- `npm run typecheck`: TypeScript kontrolü

## Kod Stili Kuralları
- ES modules (import/export) kullan
- Destructure imports when possible
- Dosya başına maksimum 300 satır
```

Token Verimliliği Kuralları

Metrik	Önerilen Değer
Maksimum talimat sayısı	150-200
İdeal satır sayısı	~60
Maksimum satır sayısı	300

Önemli: Fazla talimat kaliteyi düşürür!

Progressive Disclosure ile @ Import

markdown

Quick Reference

Temel komutlar burada

Detailed Documentation

- Proje yapısı: @README.md
- API referansı: @docs/api.md
- Stil rehberi: @docs/style-guide.md
- Veritabanı şeması: @docs/schema.md

Hızlı Bellek Ekleme

ile başlayarak anında talimat ekleyin:

```
# Always use ruff for linting
# Prefer async/await over threading
# Never commit .env files
# Test coverage must be >80%
```

Bölüm 13: Slash Commands ve Custom Workflow'lar

Slash Command Yapısı

Komutlar `.claude/commands/` klasöründe markdown dosyaları olarak tanımlanır:

yaml

```
---
allowed-tools: Bash(git add:*), Bash(git status:*), Bash(git commit:*)
argument-hint: [message]
description: Create a git commit
---
```

```
## Context
- Current git status: !`git status`
- Current git diff: !`git diff HEAD`
- Current branch: !`git branch --show-current`
```

```
## Task
Create a commit with message: $ARGUMENTS
```

Built-in Slash Commands

Komut	Açıklama
<code>/clear</code>	Context'i temizle
<code>/compact [instructions]</code>	Konuşmayı sıkıştır
<code>/context</code>	Token kullanımını görselleştir
<code>/cost</code>	Maliyet istatistikleri
<code>/agents</code>	Subagent yönetimi
<code>/memory</code>	CLAUDE.md düzenleme
<code>/init</code>	Proje başlatma
<code>/review</code>	Kod incelemesi
<code>/permissions</code>	İzin yönetimi
<code>/mcp</code>	MCP durumu
<code>/model</code>	Model seçimi
<code>/sandbox</code>	Sandbox modu
<code>/rename</code>	Oturum yeniden adlandır

Örnek: commit-push-pr.md

```
yaml
---
allowed-tools: Bash, Read, Edit
argument-hint: [PR title]
description: Commit changes, push branch, create PR
---

## Steps
1. Stage all changes: `git add -A`
2. Create descriptive commit message based on diff
3. Push to current branch
4. Create PR with title: $ARGUMENTS

## Rules
- Commit message must follow conventional commits
- PR description must include "What" and "Why" sections
```

Örnek: fix-github-issue.md

```
yaml
---
allowed-tools: Bash, Read, Write, Edit
argument-hint: [issue number]
description: Fix a GitHub issue
---

GitHub issue'yu analiz et ve çöz: $ARGUMENTS

1. `gh issue view $ARGUMENTS` ile issue detaylarını al
2. Codebase'de ilgili dosyaları ara
3. Gerekli değişiklikleri implement et
4. Test yaz ve çalıştır
5. Descriptive commit oluştur
6. Push et ve PR aç
```


Inline Bash ile Ön Hesaplama

Model ile gereksiz konuşmayı azaltmak için:

```
yaml
---
description: Analyze recent changes
---

Son değişiklikleri analiz et

## Git Status
!`git status`

## Recent Commits
!`git log --oneline -10`

## Diff Summary
!`git diff HEAD~5 --stat`

Yukarıdaki çıktıya göre kod kalitesi değerlendirmesi yap.
```

Bölüm 14: Maliyet Optimizasyonu ve Token Yönetimi

Maliyet İstatistikleri

Metrik	Değer
Ortalama günlük maliyet	\$6/geliştirici
%90'ın altında	\$12/gün
Aylık (Sonnet 4.5)	\$100-200/geliştirici
Yoğun Opus kullanımı	\$1,000+/gün

Token Fiyatlandırması

Model	Input (1M token)	Output (1M token)
Sonnet 4.5	\$3.00	\$15.00
Opus 4.5	\$15.00	\$75.00
200K üstü input	2x fiyat	Normal

Maliyet Faktörleri

200K Eşığı:

- 200K token üstü input'lar **2x fiyat**
- Output: \$22.50/milyon token (200K üstü)

Oturum Maliyetleri:

Oturum Türü	Tahmini Maliyet
Odaklı kısa oturum	\$0.60-\$1.20
Orta karmaşıklık	\$5-\$15
Tam özellik geliştirme	\$42.25
Uzun otonom çalışma	\$100+

Tasarruf Yöntemleri

1. Extended Thinking Dengesi:

- +%20-50 token
- AMA: Dramatik olarak daha iyi sonuçlar

2. Batch API:

- %50 indirim
- Acil olmayan işler için ideal

3. Prompt Caching:

- %90'a kadar tasarruf
- Tekrarlayan context için

4. Context Yönetimi:

- `/context` ile düzenli izleme
- %80'de yeni oturum
- Kullanılmayan MCP'leri kaldırma

Topluluk Hibrit Yaklaşımı

Araç	Aylık Maliyet	Kullanım
Cursor	\$20	Günlük kodlama
Claude Code	~\$100	Karmaşık görevler
Toplam	\$120	3x üretkenlik

Bölüm 15: Enterprise Adoption ve Vaka Çalışmaları

Enterprise Müşteriler

Şirket	Sektör
Netflix	Streaming
Spotify	Müzik
KPMG	Danışmanlık
L'Oreal	Kozmetik
Salesforce	CRM
Accenture	Danışmanlık

Accenture Ortaklığı (9 Aralık 2025)

Anthropic, Accenture ile ~30,000 profesyoneli eğitmek için ortaklık kurdu.

Kapsam:

- Claude Code eğitimi
- Best practices
- Enterprise entegrasyonu

Rakuten Vaka Çalışması

Metrik	Önce	Sonra	İyileşme
Feature teslim süresi	24 gün	5 gün	%79 azalma
Paralel oturum	1	20+	20x

Enterprise Güvenlik Özellikleri

Özellik	Açıklama
SOC 2 Type 2	Güvenlik sertifikası
ISO 27001	Bilgi güvenliği standardı
SSO	Single Sign-On
Admin API'leri	Merkezi yönetim
Zero-Data-Retention	Veri saklamama seçeneği

Claude Code in Slack (8 Aralık 2025)

Slack entegrasyonu ile @Claude kullanarak kanallarda Claude Code çağrılabilir.

Bölüm 16: Python ve LLM Projeleri için Yapılandırma

Python CLAUDE.md Şablonu

markdown

CLAUDE.md — Modern Python Project

Environment

- Python 3.11+
- Package Manager: uv (pip'ten 10-100x hızlı)
- Local setup: `uv sync`

Commands

- Lint: `ruff check . --fix`
- Format: `ruff format .`
- Type check: `mypy src/ --strict`
- Test: `pytest --cov=src --cov-report=html`
- All checks: `ruff check . && ruff format . && mypy src/ && pytest`

Code Style

- Python: ruff, black, mypy strict
- Line length: 88 (Black default)
- Type hints: Tüm fonksiyonlar için zorunlu
- Docstrings: Google style (Args, Returns, Raises)

Testing

- Minimum coverage: 80%
- Her bug fix için test ekle
- Unit testleri integration testlerine tercih et

File Boundaries

- Safe to edit: /src/, /tests/, /docs/
- Never touch: /venv/, /__pycache__/, /.pytest_cache/, .env

Git & CI

- Branch from `main`; squash-merge via PR
- Required checks: tests, lint, type-check

Python Slash Commands

test.md:

yaml

allowed-tools: Bash, Read, Edit

argument-hint: [test-pattern]

description: Run Python tests with optional pattern

Create pytest tests for: \$ARGUMENTS

Requirements:

- Test normal operation with valid data
- **Test edge cases:** empty inputs, missing values
- Test expected exceptions
- Use pytest fixtures

lint.md:

yaml

allowed-tools: Bash(ruff:*), Bash(mypy:*)

description: Run Python linting and formatting

Run code quality checks:

1. `ruff check . --fix`
2. `ruff format .`
3. `mypy src/ --strict`

Report remaining issues.

Python Subagents

test-runner.md:

yaml

name: test-runner

description: Python test specialist

tools: Read, Bash, Edit

model: inherit

You are a Python testing expert.

Commands:

- `pytest -v --tb=short`
- `pytest --cov=src --cov-report=term-missing`
- `pytest -x` (stop on first failure)
- `pytest --lf` (last failed)

Always verify fixes by re-running tests.

Python Hooks

PostToolUse: Otomatik Ruff Formatting:

```
json

{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "file_path=$(jq -r '.tool_input.file_path'); if echo \"$file_path\" | grep -q '\\.py$'; then ruff check --fix \"$file_path\"; fi"
      ]
    ]
  }
}
```

LLM/AI Projeleri için Context Yönetimi

Kritik Kural: 60k token veya %30 context'te `/clear` kullanın.

Document & Clear Pattern:

1. Claude'un ilerlemeyi `.md` dosyasına yazmasını isteyin
2. `/clear` ile context'i temizleyin

3. Yeni session'da `.md` dosyasını okutun

4. Çalışmaya devam edin

AI/ML CLAUDE.md Şablonu

markdown

AI/ML Project Configuration

Commands

- `python train.py`: Model eğitimi
- `python eval.py --model checkpoint.pt`: Model değerlendirme
- `pytest tests/`: Test suite
- `jupyter lab`: Jupyter ortamı

Project Structure

- `models/`: Model mimarileri
- `data/`: Dataset loaders
- `configs/`: Hyperparameter configs (YAML)
- `notebooks/`: Experiment notebooks

ML Guidelines

- Tensor şekilleri için type hints kullan
- Tüm deneyleri MLflow'a logla
- Model testleri için fixtures kullan
- PyTorch Lightning tercih et

Common Issues

- OOM errors: batch_size azalt
- Slow training: DataLoader num_workers kontrol et
- Import errors: venv aktifleştir

Bölüm 17: Ürün Lansmanları ve Yol Haritası

VS Code Extension Beta (29 Eylül 2025)

Özellikler:

- Real-time inline diff'ler
- Sidebar panelleri

- Extended thinking toggle'ları
- Tam IDE entegrasyonu

Web Lansmanı (20 Ekim 2025)

claude.ai/code Özellikleri:

- İzole sandbox ortamı
- GitHub entegrasyonu
- iOS mobil uygulama desteği
- 5-10 paralel oturum

Opus 4.5 Limitleri Kaldırıldı (Kasım 2025)

Önceden Opus 4.5 kullanımında **rate limit** vardı, artık kaldırıldı.

/sandbox Özelliği

İzin Prompt'larında %84 Azalma:

```
bash
```

```
/sandbox
```

Bu komut, güvenli bir sandbox ortamı oluşturur ve izin istemlerini minimize eder.

Gelecek Yol Haritası

Özellik	Beklenen Zaman
Agent SDK genişletmesi	Q1 2026
Multi-modal kod analizi	Q2 2026
Daha fazla IDE entegrasyonu	Devam eden
Enterprise özellikler	Devam eden

Bölüm 18: Topluluk Kaynakları ve Öğrenme

Resmi Kaynaklar

Kaynak	URL
Best Practices	anthropic.com/engineering/claude-code-best-practices
Dokümantasyon	docs.anthropic.com/claude-code
GitHub	github.com/anthropics/claude-code
DeepLearning.AI Kursu	claude-code-a-highly-agentic-coding-assistant

GitHub Repositories

Repo	Açıklama
awesome-claude-code	Kapsamlı kaynak listesi
claude-code-tips	40+ ipucu koleksiyonu
claude-code-thinking-patch	Extended thinking patch
claude-context-local	Local context yönetimi

YouTube Eğitimleri

Kanal	İçerik	Süre
CS Dojo	Claude Code Masterclass	20 dk
Data Science with Harshit	Setup ve MCP	27 dk
Krish Naik	Getting Started	16 dk
KodeKloud	Beginners Guide	Hands-on

Podcast'ler

Podcast	Konuklar	Konu
Latent Space	Boris + Cat	Deep dive
Every.to	Boris	How to use
Peterman Pod	Ekip	%70 productivity

Topluluk Platformları

Platform	Kullanım
Reddit r/ClaudeAI	Tartışmalar
Hacker News	Teknik analizler
Discord	Canlı destek
aitmpl.com	400+ component marketplace

Advent of Claude 2025

Ado Kukic'in 31 günlük ipucu serisi:

Hafta	Konular
1. Hafta	Temel özellikler
2. Hafta	Hooks ve automation
3. Hafta	Subagents ve MCP
4. Hafta	İleri düzey teknikler

Bölüm 19: Tam Proje Yapısı ve Dosya Organizasyonu

Önerilen Proje Yapısı

```
your-project/
├── .claude/
│   ├── agents/
│   │   ├── code-reviewer.md
│   │   ├── test-runner.md
│   │   ├── python-pro.md
│   │   ├── security-auditor.md
│   │   └── debugger.md
│   ├── commands/
│   │   ├── test.md
│   │   ├── lint.md
│   │   ├── review.md
│   │   ├── commit-push-pr.md
│   │   └── brainstorm.md
│   ├── skills/
│   │   └── ml-development/
│   │       └── SKILL.md
│   ├── settings.json      # Proje ayarları (Git'e commit)
│   └── settings.local.json # Kişisel ayarlar (.gitignore)
├── .mcp.json              # MCP server yapılandırması
├── CLAUDE.md              # Proje belleği (Git'e commit)
├── CLAUDE.local.md        # Kişisel proje notları (.gitignore)
├── HANDOFF.md             # Session transfer dökümanı
├── .pre-commit-config.yaml
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── claude-action.yml
├── pyproject.toml
├── src/
│   └── your_package/
│       ├── __init__.py
│       └── main.py
├── tests/
│   ├── conftest.py
│   ├── unit/
│   └── integration/
└── docs/
```

Tam settings.json Örneği

json

```
{
  "permissions": {
    "allow": [
      "Bash(uv run pytest:*)",
      "Bash(uv sync:*)",
      "Bash(ruff:*)",
      "Bash(mypy:*)",
      "Bash(git diff:*)",
      "Bash(git status:*)",
      "Bash(git log:*)",
      "Bash(git add:*)",
      "Bash(git commit:*)",
      "Bash(git push:*)",
      "Bash(docker build:*)",
      "Bash(docker run:*)",
      "Read(~/.zshrc)",
      "Read(~/.gitconfig)"
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Bash(curl:*)",
      "Bash(wget:*)",
      "Bash(sudo:*)",
      "Read(/.env)",
      "Read(/.env.*)",
      "Read(/secrets/**)",
      "Read(**/*.key)",
      "Read(**/*.pem)",
      "Write(/.env)",
      "Write(/secrets/**)"
    ],
    "defaultMode": "default"
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "file_path=$(jq -r '.tool_input.file_path'); if echo \"$file_path\" | grep -q '\\.py$'; then ruff check --fix \"$file_path\"; fi"
          }
        ]
      }
    ]
  }
}
```

```
    }
  ],
  "Stop": [
    {
      "matcher": "",
      "hooks": [
        {
          "type": "command",
          "command": "echo \"$(date): Claude task completed\" >> ~/.claude/activity.log"
        }
      ]
    }
  ],
  "SessionStart": [
    {
      "matcher": "",
      "hooks": [
        {
          "type": "command",
          "command": "echo \"$(date): New session in $(pwd)\" >> ~/.claude/activity.log"
        }
      ]
    }
  ],
  "model": "opus"
}
```

Tam .mcp.json Örneği

json

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    },
    "slack": {
      "type": "http",
      "url": "https://slack.mcp.anthropic.com/mcp"
    },
    "context7": {
      "type": "http",
      "url": "https://context7.com/mcp"
    },
    "postgres": {
      "command": "npx",
      "args": [
        "@modelcontextprotocol/server-postgres",
        "--env",
        "DATABASE_URL=postgresql://localhost/mydb"
      ]
    }
  }
}
```

Bölüm 20: Sonuç ve Kritik Öneriler

Tüm Kaynaklardan En Önemli 10 İçgörü

#	İçgörü	Kaynak
1	Doğrulama mekanizması kurun - Kaliteyi 2-3x artırır	Boris Cherny
2	Plan mode varsayılan - Oturumların %90'ı	Ado Kukic
3	Opus 4.5 kullanın - Yavaş ama daha az yönlendirme	Boris Cherny
4	5-15+ paralel oturum - Throughput maksimize	Boris Cherny
5	CLAUDE.md yaşayan döküman - Ekiple paylaş, sürekli güncelle	Alex Albert

#	İçgörü	Kaynak
6	Context yönetimi kritik - Her 3-4 etkileşimde /context	Ado Kukic
7	Hooks deterministik kontrol - Format, güvenlik, logging	Daisy Hollman
8	Subagent'lar uzmanlaşma - Bağımsız context, paralel çalışma	Sid Bidasaria
9	MCP token farkındalığı - Kullanılmayanları kaldır (%8-30 tüketim)	Ado Kukic
10	Extended thinking karmaşıklık için - "ultrathink" 32K token	Ado Kukic

En Etkili 5 Uygulama

1. CLAUDE.md dosyası zorunludur

- WHAT, WHY, HOW yapısı
- 60 satır ideal, 300 maksimum
- Ekiple paylaş, haftada birkaç kez güncelle

2. Plan mode ile başla

- Shift+Tab 2x
- 2-3x başarı artışı
- Plan beğenilince auto-accept

3. Doğrulama feedback loop

- Test suite çalıştırma
- Browser test
- Type checking
- 2-3x kalite artışı

4. Context yönetimi agresif

- %80'de /clear
- Document & Clear pattern
- HANDOFF.md kullan

5. Opus modelini kullan

- Daha yavaş ama daha az yönlendirme
- Tool kullanımında çok daha başarılı

- Sonuçta daha hızlı

Kaçınılması Gerekenler

✗ Yapma

✓ Yap

--dangerously-skip-permissions

/permissions ile whitelist

/compact kullan

/clear + HANDOFF.md

Tüm MCP'leri aktif tut

Sadece 2-3 hedefli MCP

Belirsiz talimatlar

Spesifik, ölçülebilir

Planlamayı atla

Plan mode ile başla

Tek uzun session

Düzenli /clear

Token limitini aşmayı bekle

%80'de yeni oturum

Günlük Rutin Önerisi

Sabah:

1. /cost ile önceki günün maliyetini kontrol et
2. HANDOFF.md oku (varsa)
3. /model opus ile model seç
4. Plan mode'da günün hedeflerini belirle

Gün İçi: 5. Her büyük görev için yeni session 6. Her 3-4 etkileşimde /context 7. %80'de /clear + HANDOFF.md 8. Doğrulama mekanizmasını kullan

Akşam: 9. HANDOFF.md güncelle 10. CLAUDE.md'ye öğrenilen dersleri ekle 11. /cost ile günlük maliyeti kontrol et

Gelecek Tahminleri

Tahmin

Kaynak

Zaman

Çoğu şirket %90 AI-generated kod

Mike Krieger

1 yıl

Darboğaz: code-writing → decision-making

Mike Krieger

Şimdi

Tahmin	Kaynak	Zaman
Agent SDK non-coding uygulamalar	Sid Bidasaria	6 ay
Skills/plugins genişleme	Daisy Hollman	Devam eden

Kaynaklar ve Referanslar

Resmi Kaynaklar

- <https://anthropic.com/engineering/claude-code-best-practices>
- <https://docs.anthropic.com/claude-code>
- <https://github.com/anthropics/claude-code>

Ekip Üyeleri Twitter

- Boris Cherny: @bcherny
- Sid Bidasaria: @sidbidasaria
- Cat Wu: @_catwu
- Daisy Hollman: @The_Whole_Daisy
- Alex Albert: @alexalbert__
- Mike Krieger: @mikeyk
- Amanda Askill: @AmandaAskill
- Jan Leike: @janleike

Topluluk

- Ado Kukic: @adocomplete (Advent of Claude 2025)
- Thariq: @trq212 (Spec-based workflow)
- Reddit: r/ClaudeAI
- GitHub: ykdojo/claude-code-tips

Podcast'ler

- Latent Space: Claude Code Deep Dive
- Every.to: How to Use Claude Code
- Peterman Pod: 70% Productivity Increase

Bu rehber, 12 Anthropic mühendisi ve topluluk liderinin tüm sosyal medya, YouTube, Reddit, makale ve podcast içeriklerinin mikroskopik analizinden derlenmiştir. Günlük geliştirme iş akışınızı önemli ölçüde iyileştirecek kapsamlı bir kaynak olarak tasarlanmıştır.

Son Güncelleme: Ocak 2026 **Versiyon:** 3.0 - Eksiksiz Türkçe Rapor **Toplam Kelime:** ~8,000+